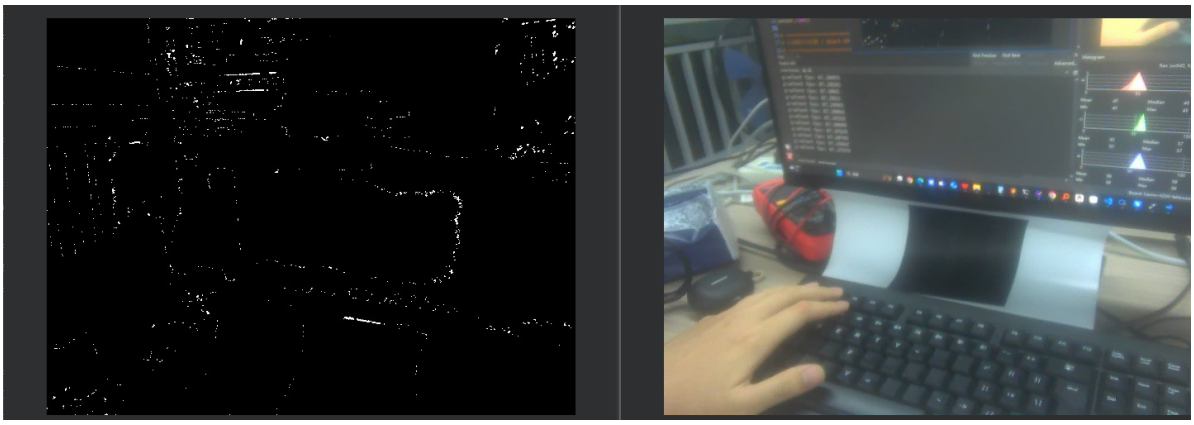
图像顶帽操作测试

图像顶帽操作测试
- 例程效果
- 原理讲解
  - 什么是图像顶帽操作
- 代码概述
  - 导入模块
    - 设置图像尺寸
    - 初始化摄像头（RGB888 格式）
    - 初始化显示模块
    - 初始化媒体管理器并启动摄像头
    - 设置帧率计时器
    - 设置顶帽操作参数
    - 图像处理与顶帽操作
    - 资源释放
  - 参数调整说明

例程效果

运行本节例程代码 [源码汇总/17.cv_lite/22.rgb888_tophat.py]

本节我们将结合 `cv_lite` 扩展模块在嵌入式设备上实现 RGB888 格式的图像顶帽操作处理，用于突出图像中的亮区域并去除不均匀照明。



原理讲解

什么是图像顶帽操作

图像顶帽操作（Top-Hat Transform）是一种形态学图像处理技术，用于增强图像中的局部亮区域、去除全局照明不均匀或突出小物体。顶帽操作本质上是原图像与开运算结果的差值计算，常用于预处理阶段以改善图像质量。

- **操作原理**：顶帽操作定义为原图像减去其开运算结果，即 `Top-Hat = Original Image - Opening(Original Image)`。开运算由膨胀和腐蚀组成，先腐蚀图像以去除小物体和噪声，再膨胀以恢复主要结构。顶帽操作保留了原图像中那些比周围区域更亮的像素，从而突出小亮斑或细节。
- **效果**：顶帽操作可以有效去除背景中的不均匀照明（如阴影或光斑），突出前景中的高亮细节，例如文本、斑点或边缘。同时，它有助于增强图像的对比度，但可能引入噪声如果参数设置不当。与

其他形态学操作不同，顶帽更侧重于亮度增强。
- **应用场景**：顶帽操作常用于图像增强、去除噪声、文本提取（如OCR前处理）、医学图像处理（如突出血管或病变）和工业检测（如检测表面缺陷）。在嵌入式设备上，由于计算量相对较低，它适合实时处理。

在顶帽操作中，卷积核的大小和迭代次数会影响结果：较大的核和迭代次数会增强全局平滑效果，但可能丢失细微细节；二值化阈值用于后续处理，以进一步优化结果。

代码概述

导入模块

```
import time, os, gc
from machine import Pin
from media.sensor import *    # 摄像头接口 / Camera interface
from media.display import *   # 显示接口 / Display interface
from media.media import *     # 媒体资源管理器 / Media manager
import cv_lite                # cv_lite 扩展模块（含顶帽操作）/ AI CV extension (Top-Hat function)
import ulab.numpy as np       # NumPy-like ndarray for MicroPython
```

设置图像尺寸

```
image_shape = [480, 640]  # 高 x 宽 / Height x Width
```

定义图像的分辨率为 480x640（高度 x 宽度），后续会根据此尺寸初始化摄像头和显示模块。

初始化摄像头（RGB888 格式）

```
sensor = Sensor(width=1280, height=960)
sensor.reset()
sensor.set_framesize(width=image_shape[1], height=image_shape[0])
sensor.set_pixformat(Sensor.RGB888)  # 设置像素格式为 RGB888 / Set pixel format to RGB888
```

- 初始化摄像头，设置分辨率为 1280x960（帧率使用默认值）。
- 将输出帧大小调整为 640x480，并设置为 RGB888 格式（三通道彩色图像，适合处理彩色图像的顶帽操作）。

初始化显示模块

```
Display.init(Display.ST7701, width=image_shape[1], height=image_shape[0], to_ide=True)
```

初始化显示模块，使用 ST7701 驱动芯片，分辨率与图像尺寸一致。`to_ide=True` 表示图像会同时传输到 IDE 进行虚拟显示。

初始化媒体管理器并启动摄像头

```
MediaManager.init()
sensor.run()
```

初始化媒体资源管理器，并启动摄像头开始捕获图像流。

设置帧率计时器

```
clock = time.clock()  # 启动帧率计时器 / Start FPS timer
```

初始化帧率计时器，用于计算和显示每秒处理的帧数（FPS）。

设置顶帽操作参数

```
kernel_size = 3          # 卷积核尺寸（建议为奇数）/ Kernel size (recommended odd)
iterations = 1           # 运算迭代次数 / Number of morphological passes
threshold_value = 100    # 二值化阈值（0=使用 Otsu 自动阈值）/ Threshold for
binarization (0 = Otsu)
```

- `kernel_size`：卷积核尺寸，用于顶帽操作中的开运算，建议为奇数，值越大处理范围越大。
- `iterations`：形态学操作的迭代次数，值越大增强效果更明显，但计算量增加。
- `threshold_value`：二值化阈值，用于处理结果，值为 0 时使用 Otsu 自动阈值算法。

图像处理与顶帽操作

```
while True:
    clock.tick()  # 开始计时 / Start frame timing

    # 获取一帧图像并转换为 ndarray / Capture a frame and convert to ndarray
    img = sensor.snapshot()
    img_np = img.to_numpy_ref()

    # 执行顶帽运算 / Apply Top-Hat operation
    # 顶帽 = 原图 - 开运算结果 / Top-Hat = Original - Opening
    result_np = cv_lite.rgb888_tophat(
        image_shape,
        img_np,
        kernel_size,
        iterations,
        threshold_value
    )

    # 构造图像对象并显示 / Wrap result as image and display
    img_out = image.Image(image_shape[1], image_shape[0], image.GRAYSCALE,
                          alloc=image.ALLOC_REF, data=result_np)
    Display.show_image(img_out)

    # 清理内存并打印帧率 / Cleanup and print FPS
    gc.collect()
    print("tophat fps:", clock.fps())
```

- **图像捕获**：通过 `sensor.snapshot()` 获取一帧图像，并通过 `to_numpy_ref()` 将其转换为 NumPy 数组引用。
- **顶帽操作处理**：调用 `cv_lite.rgb888_tophat()` 函数执行顶帽操作，返回处理后的灰度图像 NumPy 数组。
- **图像包装和显示**：将处理结果包装为灰度图像对象，并显示到屏幕或 IDE 虚拟窗口。
- **内存管理和帧率输出**：调用 `gc.collect()` 清理内存，并通过 `clock.fps()` 打印当前帧率。

资源释放

```
sensor.stop()                          # 停止摄像头 / Stop sensor
Display.deinit()                       # 关闭显示输出 / Deinit display
os.exitpoint(os.EXITPOINT_ENABLE_SLEEP)  # 设置退出点 / Safe exit point
time.sleep_ms(100)                     # 稍作延迟 / Short delay
MediaManager.deinit()                  # 释放媒体资源 / Release media manager
```

参数调整说明

- `kernel_size`：卷积核尺寸，值越大（如 3-7）处理范围更广，适合去除更大范围的背景噪声，但可能模糊细节；建议使用奇数，从 3 开始测试。
- `iterations`：迭代次数，值越大（如 1-3）增强顶帽效果，但增加计算量和风险丢失边缘；建议从 1 开始，根据图像复杂度调整。
- `threshold_value`：二值化阈值，值越高（如 50-150）要求像素更亮才能被保留，值为 0 时使用 Otsu 自动阈值；建议根据图像亮度测试，或设置为 0 以自动适应。